



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“Registro_Contactos”

Curso: Programación II

Docente: BRENO SOLIM LUQUE ZÚÑIGA

- **Jiménez Romero, Josué Andre (2022074259)**

Tacna – Perú
2026



INDICE

Contenido

INDICE	2
I. INFORMACIÓN GENERAL	3
- Objetivos	3
- Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEORICO	3
III. PROCEDIMIENTO	4
IV. ANALISIS E INTERPRETACION DE RESULTADOS	14
CONCLUSIONES	14
RECOMENDACIONES	14



INFORME DE LABORATORIO

TEMA: Registro Contactos

I. INFORMACIÓN GENERAL

- **Objetivos:** Desarrollar una aplicación de escritorio integral utilizando C# y Windows Forms, aplicando los principios de Programación Orientada a Objetos (POO). El sistema debe gestionar un CRUD completo (Crear, Leer, Actualizar, Eliminar) de contactos personales, utilizando colecciones dinámicas y filtrado de datos en tiempo real.
- **Equipos, materiales, programas y recursos utilizados:**
 - IDE de desarrollo (Visual Studio 2022 o similar).
 - Lenguaje de programación C# (.NET Framework / .NET Core).
 - Computadora con sistema operativo Windows y al menos 4GB de RAM.

II. MARCO TEORICO

El desarrollo de la práctica se basó en la gestión de datos en memoria y la interacción de controles avanzados:

- **Parte I: Encapsulamiento y POO:**

Se implementó una clase modelo denominada Contacto, la cual encapsula los atributos del usuario (ID, Nombre, Apellido, Teléfono, Dirección y Género) mediante propiedades auto-implementadas, separando la definición de los datos de la interfaz lógica.

- **Parte II: Colecciones Dinámicas (List):**

A diferencia de los arreglos estáticos, se utilizó List<Contacto> para permitir un almacenamiento flexible que crece dinámicamente conforme se agregan nuevos registros, facilitando la gestión de la memoria.



- **Parte III: Tecnología LINQ (Language Integrated Query):**

Se aplicaron consultas LINQ mediante expresiones Lambda para realizar búsquedas complejas y filtrado dinámico sobre la lista de contactos, optimizando la recuperación de información sin recurrir a ciclos iterativos manuales extensos.

- **Parte IV: Eventos y Controles de Interfaz:**

Se trabajó con el control DataGridView para la visualización tabular, ComboBox para la selección de género y eventos de manejo de estado como CellClick (para selección de filas) y TextChanged (para búsqueda reactiva).

III. PROCEDIMIENTO

Figura 1

Diseño de formulario Registro_ Contactos

The image shows a software interface for managing contacts. The window is titled 'CONTACTOS' and has a close button in the top right corner. On the left side, there are input fields for 'ID Contacto', 'Nombre', 'Apellido', 'Teléfono', and 'Dirección'. Below these is a 'Género' dropdown menu. To the right of these fields is a search bar labeled 'Buscar'. At the bottom of the form, there are four buttons: 'Agregar' (green), 'Actualizar' (blue), 'Eliminar' (red), and 'Limpiar' (yellow). The 'Agregar' button is currently selected, indicated by a dashed border and a small square handle.



Figura 2

Botón agregar

```
2 referencias
private void btnAgregar_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtNombre.Text) ||
        string.IsNullOrEmpty(txtApellido.Text) ||
        string.IsNullOrEmpty(txtTelefono.Text))
    {
        MessageBox.Show("Por favor, llene al menos el Nombre, Apellido y Teléfono.", "Error de validación", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    Contacto nuevoContacto = new Contacto
    {
        ContactID = contadorId,
        Nombre = txtNombre.Text,
        Apellido = txtApellido.Text,
        Telefono = txtTelefono.Text,
        Direccion = txtDireccion.Text,
        Genero = cmbGenero.SelectedItem.ToString()
    };

    listaContactos.Add(nuevoContacto);
    contadorId++;
    ActualizarDataGridView();
    LimpiarCampos();
    MessageBox.Show("Contacto agregado exitosamente.", "Éxito", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
```

Figura 3

Botón actualizar

```
private void btnActualizar_Click(object sender, EventArgs e)
{
    if (idContactoSeleccionado == -1)
    {
        MessageBox.Show("Por favor, seleccione un contacto de la lista haciendo clic sobre él.", "Atención", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (string.IsNullOrEmpty(txtNombre.Text) ||
        string.IsNullOrEmpty(txtApellido.Text) ||
        string.IsNullOrEmpty(txtTelefono.Text))
    {
        MessageBox.Show("Por favor, no deje el Nombre, Apellido o Teléfono en blanco.", "Error de validación", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    Contacto contactoAAActualizar = listaContactos.Find(c => c.ContactID == idContactoSeleccionado);

    if (contactoAAActualizar != null)
    {
        contactoAAActualizar.Nombre = txtNombre.Text;
        contactoAAActualizar.Apellido = txtApellido.Text;
        contactoAAActualizar.Telefono = txtTelefono.Text;
        contactoAAActualizar.Direccion = txtDireccion.Text;
        contactoAAActualizar.Genero = cmbGenero.SelectedItem.ToString();

        ActualizarDataGridView();
        LimpiarCampos();

        idContactoSeleccionado = -1;

        MessageBox.Show("Contacto actualizado exitosamente.", "Éxito", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}
```



Figura 4

Botón eliminar

```
2 referencias
private void btnEliminar_Click(object sender, EventArgs e)
{
    if (idContactoSeleccionado == -1)
    {
        MessageBox.Show("Por favor, seleccione un contacto de la lista haciendo clic sobre él para eliminarlo.", "Atención", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    DialogResult respuesta = MessageBox.Show("¿Está seguro de que desea eliminar este contacto de forma permanente?", "Confirmar Eliminación", MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (respuesta == DialogResult.Yes)
    {
        Contacto contactoAEliminar = listaContactos.Find(c => c.ContactID == idContactoSeleccionado);

        if (contactoAEliminar != null)
        {
            listaContactos.Remove(contactoAEliminar);

            ActualizarDataGridView();
            LimpiarCampos();

            idContactoSeleccionado = -1;

            MessageBox.Show("Contacto eliminado exitosamente.", "Éxito", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
}
```

Figura 5

Botón limpiar

```
2 referencias
private void btnLimpiar_Click(object sender, EventArgs e)
{
    LimpiarCampos();
    idContactoSeleccionado = -1;
    txtBuscar.Text = "";
    ActualizarDataGridView();
}
```

Figura 6

Función buscar (asignando como acción)

```
2 referencias
private void txtBuscar_TextChanged(object sender, EventArgs e)
{
    string palabraClave = txtBuscar.Text.ToLower();
    List<Contacto> listaFiltrada = listaContactos.FindAll(c =>
        c.Nombre.ToLower().Contains(palabraClave) ||
        c.Apellido.ToLower().Contains(palabraClave) ||
        c.Telefono.Contains(palabraClave)
    );
    dgvContactos.DataSource = null;
    dgvContactos.DataSource = listaFiltrada;
}
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

El sistema demostró una alta eficiencia en la gestión de contactos en memoria. La implementación de la lógica CRUD permitió una manipulación fluida de los datos:

- La **validación de datos** previno errores de ejecución al evitar registros nulos.
- La variable de estado `idContactoSeleccionado` permitió vincular correctamente la fila seleccionada en el `DataGridView` con el objeto real en la lista, garantizando que las actualizaciones y eliminaciones afectaran al registro correcto.
- La **búsqueda dinámica** mediante el evento `TextChanged` mejoró significativamente la experiencia de usuario (UX), permitiendo encontrar contactos instantáneamente entre múltiples registros sin necesidad de refrescar manualmente la interfaz.

V. CONCLUSIONES

- La aplicación de los principios de **POO** facilita el mantenimiento del código; al tener una clase `Contacto` independiente, es posible escalar el proyecto (por ejemplo, añadir una base de datos) con cambios mínimos en la lógica.
- El uso de **colecciones dinámicas (`List<T>`)** junto con **LINQ** reduce la complejidad ciclomática del software, haciendo que el código sea más legible, eficiente y menos propenso a errores de desbordamiento de memoria comunes en arreglos fijos.
- La correcta gestión de eventos en Windows Forms permite crear aplicaciones robustas donde la interfaz reacciona de manera coherente a las acciones del usuario, manteniendo la integridad de los datos en todo momento.



VI. RECOMENDACIONES

- **Mejora de la Arquitectura:** Considerar el patrón de diseño MVC (Modelo-Vista-Controlador) para separar aún más la lógica de eventos del formulario de la manipulación de la lista de contactos.
- **Persistencia de datos:** Para futuras versiones, se recomienda integrar una base de datos local (como SQLite o SQL Server) para que la información no se pierda al cerrar la aplicación

